

07-01 파이썬 웹크롤링 기본

웹 크롤러란?

우리가 어떤 정보를 브라우저에서만 보는 것 뿐 아니라 그 정보들을 내가 이용하기 편한 방식(ex: json)으로 로컬에 저장하고 싶을 때가 있다.

[HTTrack](#)의 경우에는 웹을 그대로 자신의 컴퓨터로 복사를 해오지만, 내가 원하는 방식으로의 가공까지는 제공해주지 않는다.

Python을 이용하면 간단한 코드 몇줄 만으로도 쉽게 웹 사이트에서 원하는 정보만을 가져올 수 있다.

웹에서 정보 가져오기

Requests

Python에는 requests라는 유명한 http request 라이브러리가 있다.

설치하기

```
pip install requests
```

pip로 간단하게 설치가 가능하다.

이용방법

Python 파일 하나(ex: parser.py)를 만들어 requests를 import 해준다.

```
# parser.py
```

```
import requests
```

```
# HTTP GET Request
```

```
req = requests.get('https://beomi.github.io/beomi.github.io_old/')
```

```
# HTML 소스 가져오기
```

```
html = req.text
```

```
# HTTP Header 가져오기
```

```
header = req.headers
```

```
# HTTP Status 가져오기 (200: 정상)
```

```
status = req.status_code
```

```
# HTTP가 정상적으로 되었는지 (True/False)
```

```
is_ok = req.ok
```

위 코드에서 우리가 사용할 것은 HTML 소스를 이용하는 것이다. 따라서 `html=req.text`를 이용한다.

BeautifulSoup

Requests는 정말 좋은 라이브러리이지만, html을 '의미있는', 즉 Python이 이해하는 객체 구조로 만들어 주지는 못한다. 위에서 `req.text`는 python의 문자열(str)객체를 반환할 뿐이기 때문에 정보를 추출하기가 어렵다.

따라서 BeautifulSoup을 이용하게 된다. 이 BeautifulSoup은 html 코드를 Python이 이해하는 객체 구조로 변환하는 Parsing을 맡고 있고, 이 라이브러리를 이용해 우리는 제대로 된 '의미있는' 정보를 추출해 낼 수 있다.

설치하기

```
pip install bs4
```

BeautifulSoup을 직접 쳐서 설치하는 것도 가능하지만, bs4라는 wrapper라이브러리를 통해 설치하는 방법이 더 쉽고 안전하다.

이용방법

위에서 이용한 parser.py파일을 좀 더 다듬어 보자.

```
# parser.py
```

```
import requests
```

```
from bs4 import BeautifulSoup
```

```
# HTTP GET Request
```

```
req = requests.get('http://movie.naver.com/movie/point/af/list.nhn')
```

```
# HTML 소스 가져오기
```

```
html = req.text
```

```
# BeautifulSoup으로 html소스를 python객체로 변환하기
```

```
# 첫 인자는 html소스코드, 두 번째 인자는 어떤 parser를 이용할지 명시.
```

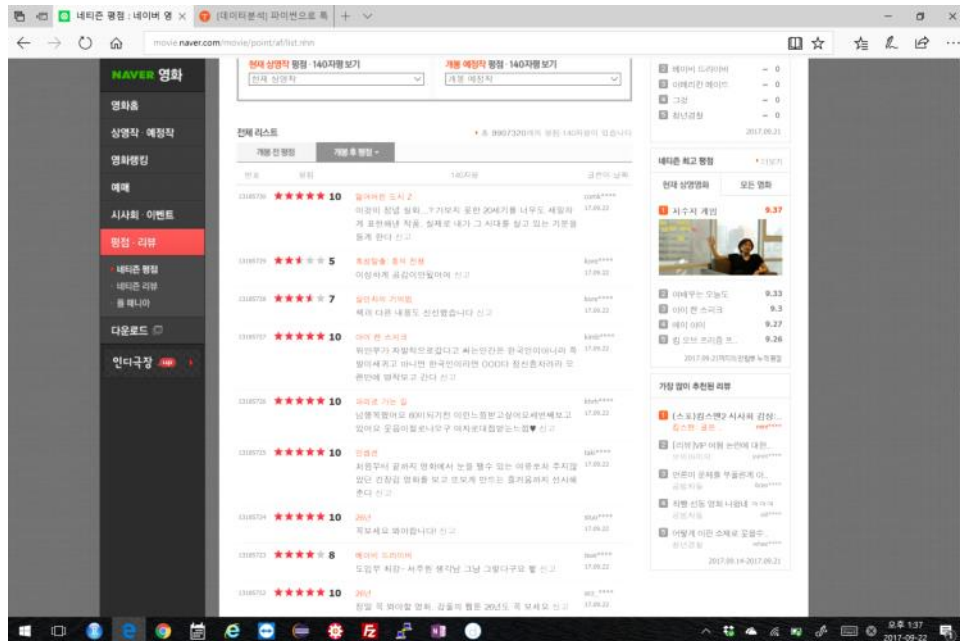
```
# 이 글에서는 Python 내장 html.parser를 이용했다.
```

```
soup = BeautifulSoup(html, 'html.parser')
```

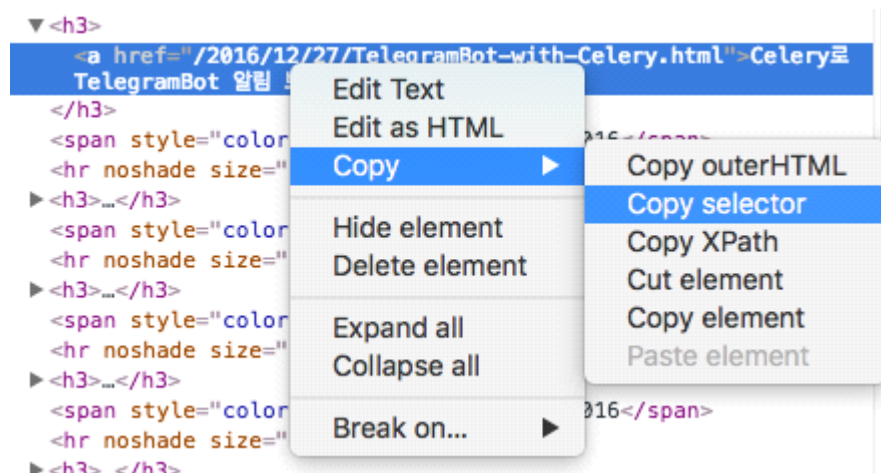
이제 soup 객체에서 원하는 정보를 찾아낼 수 있다.

BeautifulSoup에서는 여러가지 기능을 제공하는데, 여기서는 select를 이용한다. select는 CSS Selector를 이용해 조건과 일치하는 모든 객체들을 List로 반환해준다.

예시로 이 블로그의 모든 제목을 가져와 보도록 하자.



크롬에 내장된 검사도구(요소 위에서 우측 클릭 후 검사)를 이용해보면 현재 title은 a 태그로 구성되어있다는 것을 알 수 있다. 이 상황에서 모든 a 태그를 가져올 수도 있지만, 보다 정확하게 가져오기 위해 CSS Selector를 확인해 보자.



확인해보니 아래와 같은 코드가 나왔다.

```
body > h3:nth-child(4) > a
```

하지만 :nth-child(4) 등이 붙어있는 것으로 보아 현재 요소를 '정확하게' 특정하고 있기 때문에, 좀 더 유연하게 만들어 주기 위해 아래와 같이 selector를 바꿔준다.(위 코드는 단 하나의 링크만을 특정하고, 아래 코드는 css selector에 일치하는 모든 요소를 가리킨다.)

```
h3 > a
```

이제 parsing.py파일을 더 다듬어 보자.

```
# parser.py
import requests
from bs4 import BeautifulSoup
req = requests.get('https://beomi.github.io/beomi.github.io_old/')
html = req.text
soup = BeautifulSoup(html, 'html.parser')
# CSS Selector를 통해 html요소들을 찾아낸다.
my_titles = soup.select(
    'h3 > a'
)
```

위 코드에서 my_titles는 string의 list가 아니라 soup객체들의 list이다. 따라서 태그의 속성들도 이용할 수 있는데, a 태그의 경우 href속성이 대표적인 예시다.

soup객체는 <태그>로 구성된 요소를 Python이 이해하는 상태로 바꾼 것이라 볼 수 있다. 따라서 여러가지로 조작이 가능하다.

```
# parser.py
import requests
from bs4 import BeautifulSoup
req = requests.get('https://beomi.github.io/beomi.github.io_old/')
html = req.text
soup = BeautifulSoup(html, 'html.parser')
my_titles = soup.select(
    'h3 > a'
)
```

my_titles는 list 객체

for title in my_titles:

 # Tag안의 텍스트

 print(title.text)

 # Tag의 속성을 가져오기(ex: href속성)

 print(title.get('href'))

위와 같이 코드를 처리할 경우 a 태그 안의 텍스트와 a 태그의 href속성의 값을 가져오게 된다. 위 코드에서 title 객체는 python의 dictionary와 같이 태그의 속성들을 저장한다. 따라서 title.get('속성이름')나 title['속성이름']처럼 이용할 수 있다.

select를 통해 요소들을 가져온 이후에는 각자가 생각하는 방식으로 python코드를 이용해 저장하면 된다.

정리 예제

아래 코드는 크롤링한 데이터를 Python파일과 같은 위치에 result.json을 만들어 저장하는 예제다.

```
# parser.py
import requests
from bs4 import BeautifulSoup
import json
import os
# python파일의 위치
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
req = requests.get('https://beomi.github.io/beomi.github.io_old/')
html = req.text
soup = BeautifulSoup(html, 'html.parser')
my_titles = soup.select(
    'h3 > a'
)
data = {}
for title in my_titles:
    data[title.text] = title.get('href')
with open(os.path.join(BASE_DIR, 'result.json'), 'w+') as json_file:
    json.dump(data, json_file)
```

07-02 네이버 뉴스 크롤링하기

네이버 뉴스 크롤링하기

네이버포털의 뉴스 기사를 긁어 오는 것을 먼저 연습하도록 하겠습니다.

```
1  """네이버 뉴스 기사 웹 크롤러 모듈"""
2
3  from bs4 import BeautifulSoup
4  import urllib.request
5
6  # 출력 파일 명
7  OUTPUT_FILE_NAME = 'output.txt'
8  # 긁어 올 URL
9  URL = 'http://news.naver.com/main/read.nhn?mode=LSD&mid=shm&sid1=103&oid=055'W
10     '&aid=0000445667'
11
12
13  # 크롤링 함수
14  def get_text(URL):
15      source_code_from_URL = urllib.request.urlopen(URL)
16      soup = BeautifulSoup(source_code_from_URL, 'lxml', from_encoding='utf-8')
17      text = ""
18      for item in soup.find_all('div', id='articleBodyContents'):
19          text = text + str(item.find_all(text=True))
20      return text
21
22
23  # 메인 함수
24  def main():
25      open_output_file = open(OUTPUT_FILE_NAME, 'w')
26      result_text = get_text(URL)
27      open_output_file.write(result_text)
28      open_output_file.close()
29
30
31  if __name__ == '__main__':
32      main()
```

위 코드는 네이버뉴스를 긁어오기 위한 크롤러 모듈 입니다.
그럼 코드를 하나하나 살펴보도록 하겠습니다.

```

1      ""네이버 뉴스 기사 웹 크롤러 모듈""
2
3      from bs4 import BeautifulSoup
4      import urllib.request

```

크롤링을 하기 위해 두 가지 라이브러리를 임포트했습니다.

HTTP REQUEST를 보내고 HTTP RESPONSE를 받기 위한 urllib과

urllib을 통해 받은 RESPONSE(HTML)를 파싱하기 위한 BeautifulSoup 입니다.

```

6      # 출력 파일 명
7      OUTPUT_FILE_NAME = 'output.txt'
8      # 긁어 올 URL
9      URL = 'http://news.naver.com/main/read.nhn?mode=LSD&mid=shm&sid1=103&oid=055'W
10     '&aid=0000445667'

```

그 후에 기사내용을 긁어와 저장할 파일명과 긁어 올 기사의 URL주소를 상수로 할당했습니다.

URL주소를 백슬래시를 사용해 두 줄로 나누었습니다. 이는 한 행이 80자가 넘어가는 것을 방지하기 위해서 입니다.

신문기사 URL주소는 본인이 크롤링 하길 원하는 뉴스기사의 URL주소를 복사해 오시면 됩니다.



저는 네이버 뉴스에서 동영상에 포함된 일기예보 기사를 골라 복사해왔습니다.

동영상이 포함된 기사였지만 일기예보 내용이 텍스트로 본문에 포함되어 있으므로 해당 기사의 본문 내용 (텍스트)만 긁어오도록 하겠습니다.

```

13     # 크롤링 함수
14     def get_text(URL):
15         source_code_from_URL = urllib.request.urlopen(URL)

```

```

16 soup = BeautifulSoup(source_code_from_URL, 'lxml', from_encoding='utf-8')
17 text = ""
18 for item in soup.find_all('div', id='articleBodyContents'):
19     text = text + str(item.find_all(text=True))
20 return text

```

위는 해당 URL주소로 요청을 보내고 받아 기사 내용을 파싱해 하나의 문자열로 저장하는 함수입니다.

```

14 def get_text(URL):
15     source_code_from_URL = urllib.request.urlopen(URL)
16     soup = BeautifulSoup(source_code_from_URL, 'lxml', from_encoding='utf-8')

```

우선 15번째 줄에서 urlopen메소드를 통해 URL주소에 대한 요청을 보내 source_code_from_URL 변수에 그 결과를 저장했습니다.

그 후, HTML코드를 파싱하기 위해 16번째 줄에서 source_code_from_URL을 이용, BeautifulSoup객체를 생성해 soup에 할당했습니다.

BeautifulSoup객체 생성자의 2번째 인자로 'lxml'을 사용해 기존 'html'방식 대신 'lxml'방식으로 파싱을 했고, 한글 내용이 포함된 기사이기 때문에 from_encoding 키워드 인자로 'utf-8'을 넣어 'UTF-8'방식으로 인코딩을 했습니다.

'lxml'방식으로 BeautifulSoup 객체를 생성할때 오류가 난다면 'lxml' 라이브러리 또한 설치해주시길 바랍니다.

이 후, soup객체에서 원하는 부분(HTML요소)만 가지고 와야 합니다. 우선 우리가 가지고 올 기사의 본문 내용이 포함된 태그(HTML 요소)가 무엇인지를 찾아야합니다.

The image shows a web browser displaying a news article from SBS. The article title is "[날씨] 한결 선선해진 바람...주말 30도 안팎" (Weather: A refreshing breeze...weekend around 30 degrees). The article content is displayed below the video player. The developer tools on the right show the DOM tree with the 'div#articleBodyContents' element selected, which contains the main text of the article. The text discusses a weather forecast for Seoul, mentioning a high of 28 degrees and a low of 23 degrees, and a 30% chance of rain.

위 사진은 신문기사 페이지에서 크롬 개발자도구를 이용해 본문 내용이 포함된 태그를 찾은 사진입니다.

```
▼<div id="articleBodyContents" style="-webkit-tap-highlight-color: rgba(0, 0, 0, 0);"> == $0
```

```
<!-- 본문 내용 -->
"
```

```
    네, 바람이 한결 선선해졌습니다."
```

```
<br>
```

```
<br>
```

```
"그동안 힘들었던 폭염이 물러갔는데요, 시원한 단비가 내린 데다가 찬 공기가 내려왔기 때문입니다."
```

```
<br>
```

```
<br>
```

```
"폭염이 물러가고 열대야도 대부분 해소됐습니다."
```

```
<br>
```

```
<br>
```

```
"이제 큰 더위 걱정은 안 하셔도 되겠는데요, 오늘(26일) 서울의 낮 기온 28도로 정말 모처럼 30도 아래로 내려왔습니다."
```

```
<br>
```

자세히 살펴보면 본문 내용은 id가 'articleBodyContents'인 'div'클래스 안에 담겨 있음을 알 수 있습니다. 따라서 우리가 생성한 객체 soup에서 위에서의 'div'클래스를 가져오기만 하면 됩니다.

```
17     text = ""
18     for item in soup.find_all('div', id='articleBodyContents'):
19         text = text + str(item.find_all(text=True))
20     return text
```

우선 본문 내용을 저장하기 위해 text에 빈 문자열을 할당합니다.

이 후, soup객체의 find_all 메소드를 이용해 id가 'articleBodyContents'인 'div'클래스를 모두 뽑아내고, for문을 통해 뽑힌 요소 하나하나에 다시 한번 직접 접근합니다.

for문으로 뽑혀진 각 요소 item에 find_all 메소드를 사용, text 키워드 인자에 True를 넣어 텍스트 요소만 뽑아 문자열로 치환한 후, text 문자열에 이어 붙였습니다.

그 후, text를 반환했습니다.

좀 더 자세한 [BeautifulSoup](#)객체의 사용법은 공식 문서를 참조하시기 바랍니다.

본문 내용이 포함된 요소가 'div'클래스 하나인데 for문을 쓴 이유는 첫번째로는 find_all 메소드의 반환 객체가 인자로 주어진 조건을 만족하는 모든 요소들을 순환가능한 객체(그 타입이 리스트인지 아닌지는 잘 모르겠네요...)로 주어지기 때문에 사용했으며, 두번째로는 다른 신문사의 경우, 본문 내용이 동일한 id를 갖는 동일한 여러개의 태그에 나뉘어 할당되어있는 경우도 있기 때문입니다.

```
23     # 메인 함수
24     def main():
25         open_output_file = open(OUTPUT_FILE_NAME, 'w')
26         result_text = get_text(URL)
27         open_output_file.write(result_text)
28         open_output_file.close()
29
30
```



```

31     if __name__ == '__main__':
32         main()

```

위는 그 이후의 코드입니다.

메인 함수를 별도로 만들고 `__name__`을 이용해 main함수를 실행시켰습니다.

메인함수에서는 위에서 할당한 `OUTPUT_FILE_NAME`상수를 통해 txt파일을 생성하고, `get_text`함수를 사용해 기사내용을 `result_text`에 할당했습니다.

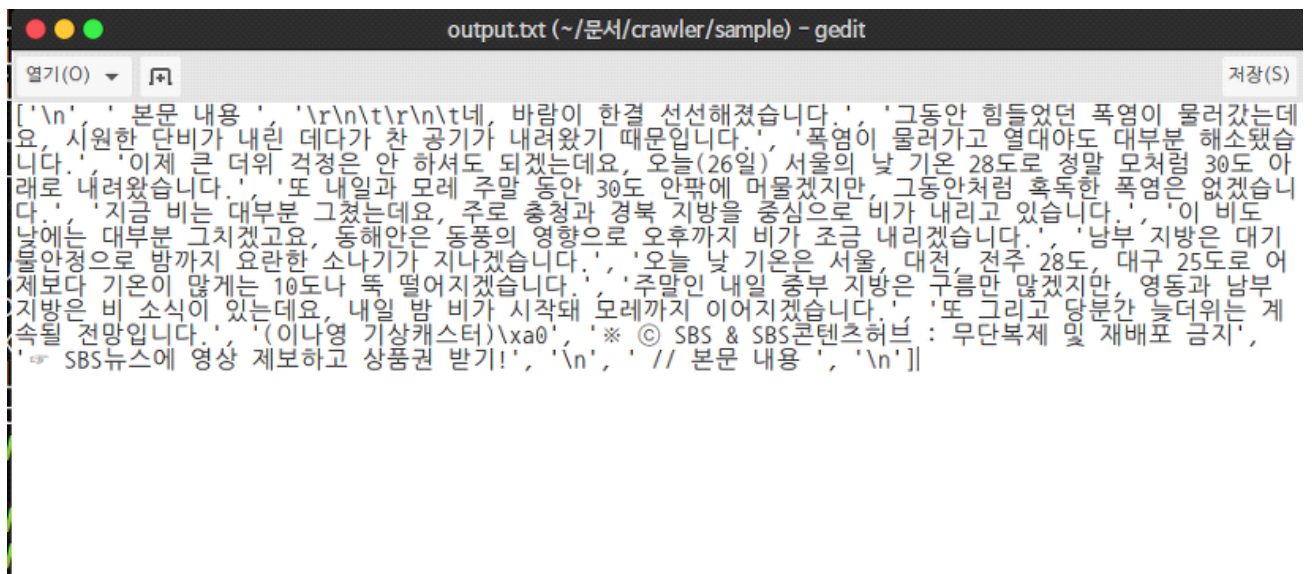
이 후, 오픈한 `output_file`에 기사를 쓰고 닫았습니다.

```

youngsik@MY-LOVELY-PC:~/문서/crawler/sample$ python3 crawler.py
youngsik@MY-LOVELY-PC:~/문서/crawler/sample$ ls
crawler.py  output.txt
youngsik@MY-LOVELY-PC:~/문서/crawler/sample$

```

터미널에서 해당 크롤러모듈을 실행한 결과 'output.txt' 파일이 생성되었으며



output.txt (~문서/crawler/sample) - gedit

```

[ '\n' , ' 본문 내용 ' , '\r\n\t\r\n\t네, 바람이 한결 선선해졌습니다. ' , ' 그동안 힘들었던 폭염이 물러갔는데
요, 시원한 단비가 내린 데다가 찬 공기가 내려왔기 때문입니다. ' , ' 폭염이 물러가고 열대야도 대부분 해소됐습
니다. ' , ' 이제 큰 더위 걱정은 안 하셔도 되겠는데요, 오늘(26일) 서울의 낮 기온 28도로 정말 모처럼 30도 아
래로 내려왔습니다. ' , ' 또 내일과 모레 주말 동안 30도 안팎에 머물겠지만, 그동안처럼 혹독한 폭염은 없겠습니
다. ' , ' 지금 비는 대부분 그쳤는데요, 주로 충청과 경북 지방을 중심으로 비가 내리고 있습니다. ' , ' 이 비도
날에는 대부분 그치겠고요, 동해안은 동풍의 영향으로 오후까지 비가 조금 내리겠습니다. ' , ' 남부 지방은 대기
불안정으로 밤까지 요란한 소나기가 지나겠습니다. ' , ' 오늘 낮 기온은 서울, 대전, 전주 28도, 대구 25도로 어
제보다 기온이 많게는 10도나 뚝 떨어지겠습니다. ' , ' 주말인 내일 중부 지방은 구름만 많겠지만, 영동과 남부
지방은 비 소식이 있는데요, 내일 밤 비가 시작돼 모레까지 이어지겠습니다. ' , ' 또 그리고 당분간 늘더위는 계
속될 전망입니다. ' , ' (이나영 기상캐스터)\xa0' , ' ※ © SBS & SBS콘텐츠허브 : 무단복제 및 재배포 금지' ,
' SBS뉴스에 영상 제보하고 상품권 받기!' , '\n' , ' // 본문 내용 ' , '\n']

```

'output.txt' 파일의 내용 또한 우리가 긁어오고자 한 신문기사의 내용이 성공적으로 긁어와졌음을 확인할 수 있습니다.

다만 'Wn', 'Wr'등의 문자와 워드클라우드 시각화에 필요 없는 특수문자의 제거가 필요해 보입니다.

따라서 텍스트 정제 모듈을 따로 만들어 추출된 기사에서 필요없는 문자를 제거하였습니다.

```

1  """ 텍스트 정제 모듈
2      영어, 특수기호 모두 제거
3      """
4
5  import re
6
7  # 입,출력 파일명
8  INPUT_FILE_NAME = 'output.txt'
9  OUTPUT_FILE_NAME = 'output_cleand.txt'
10

```

```

11
12 # 클리닝 함수
13 def clean_text(text):
14     cleaned_text = re.sub('[a-zA-Z]', '', text)
15     cleaned_text = re.sub('[W{W}W[W]W/?.,;:|W)*~`!^W-_+<>@W#$%&WWW=W(W'W"]',
16         "", cleaned_text)
17     return cleaned_text
18
19
20 # 메인 함수
21 def main():
22     read_file = open(INPUT_FILE_NAME, 'r')
23     write_file = open(OUTPUT_FILE_NAME, 'w')
24     text = read_file.read()
25     text = clean_text(text)
26     write_file.write(text)
27     read_file.close()
28     write_file.close()
29
30
31 if __name__ == "__main__":
32     main()

```

위 코드는 텍스트 정제 모듈입니다.

```

1 """ 텍스트 정제 모듈
2     영어, 특수기호 모두 제거
3     """
4
5 import re
6
7 # 입,출력 파일명
8 INPUT_FILE_NAME = 'output.txt'
9 OUTPUT_FILE_NAME = 'output_cleand.txt'
10

```

정규표현식을 활용하기 위해 're' 라이브러리를 임포트하고, 정제할 파일의 이름을 INPUT_FILE_NAME으로, 결과 파일 이름을 OUTPUT_FILE_NAME으로 할당했습니다.

```

12 # 클리닝 함수

```

```

13 def clean_text(text):
14     cleaned_text = re.sub('[a-zA-Z]', '', text)
15     cleaned_text = re.sub('[W{W}W[W]W/?.,:|W)*~`!^W-_+<>@W#$$%&WWW=W(W'W"]',
16         '', cleaned_text)
17     return cleaned_text

```

이 후, 문자열을 입력 받아 영어와 특수문자를 제거하는 클리닝 함수를 정의했습니다.
 14번째 줄은 대소문자 영어를 제거하는 코드, 15번째 줄은 특수문자를 제거하는 코드입니다.
['re'라이브러리의 자세한 사용법은 공식 문서를 참조하시기 바랍니다.](#)

```

20 # 메인 함수
21 def main():
22     read_file = open(INPUT_FILE_NAME, 'r')
23     write_file = open(OUTPUT_FILE_NAME, 'w')
24     text = read_file.read()
25     text = clean_text(text)
26     write_file.write(text)
27     read_file.close()
28     write_file.close()
29
30
31 if __name__ == "__main__":
32     main()

```

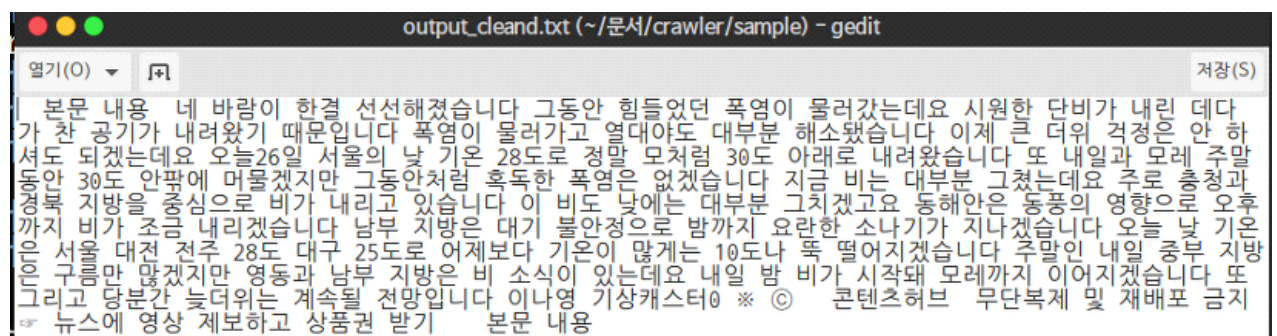
이번 모듈도 마찬가지로 main를 정의하고 __name__을 이용해 main함수를 실행했습니다.
 main함수에서 위에서 상수로 정의한 입력파일명을 이용해 입력파일에서 문자열을 읽고, clean_text함수를
 이용해 본문 내용을 정제했습니다.
 이후 출력파일명을 이용해 출력파일을 열고, 출력파일에 정제한 본문 내용을 썼습니다.

```

youngsik@MY-LOVELY-PC:~/문서/crawler/sample$ python3 cleaner.py
youngsik@MY-LOVELY-PC:~/문서/crawler/sample$ ls
cleaner.py  crawler.py  output.txt  output_cleand.txt
youngsik@MY-LOVELY-PC:~/문서/crawler/sample$

```

터미널을 통해 해당 정제모듈을 실행한 결과 'output_cleand.txt'이 생성되었고,



output_cleand.txt (~문서/crawler/sample) - gedit

본문 내용 네 바람이 한결 선선해졌습니다 그동안 힘들었던 폭염이 물러갔는데요 시원한 단비가 내린 데다
 가 찬 공기가 내려왔기 때문입니다 폭염이 물러가고 열대야도 대부분 해소됐습니다 이제 큰 더위 걱정은 안 하
 셔도 되겠는데요 오늘26일 서울의 낮 기온 28도로 정말 모처럼 30도 아래로 내려왔습니다 또 내일과 모레 주말
 동안 30도 안팎에 머물겠지만 그동안처럼 혹독한 폭염은 없겠습니다 지금 비는 대부분 그쳤는데요 주로 충청과
 경북 지방을 중심으로 비가 내리고 있습니다 이 비도 낮에는 대부분 그치겠고요 동해안은 동풍의 영향으로 오후
 까지 비가 조금 내리겠습니다 남부 지방은 대기 불안정으로 밤까지 요란한 소나기가 지나겠습니다 오늘 낮 기온
 은 서울 대전 전주 28도 대구 25도로 어제보다 기온이 많게는 10도나 뚝 떨어지겠습니다 주말인 내일 중부 지방
 은 구름만 많겠지만 영동과 남부 지방은 비 소식이 있는데요 내일 밤 비가 시작돼 모레까지 이어지겠습니다 또
 그리고 당분간 더더위는 계속될 전망입니다 이나영 기상캐스터 ※ © 콘텐츠허브 무단복제 및 재배포 금지
 뉴스에 영상 제보하고 상품권 받기 본문 내용

'output_cleand.txt' 파일을 열어 본 결과, 영어와 불필요한 문자들이 제거되었음을 확인할 수 있습니다.

07-03 동아일보, 한겨레 신문 '사드' 관련 신문기사 크롤링

동아일보, 한겨레 신문에서 '사드'관련 신문기사 크롤링하기

```
1 """ 동아일보 특정 키워드를 포함하는, 특정 날짜 이전 기사 내용 크롤러(정확도순 검색)
2     python [모듈 이름] [키워드] [가져올 페이지 숫자] [결과 파일명]
3     한 페이지에 기사 15개
4     """
5
6     import sys
7     from bs4 import BeautifulSoup
8     import urllib.request
9     from urllib.parse import quote
10
11     TARGET_URL_BEFORE_PAGE_NUM = "http://news.donga.com/search?p="
12     TARGET_URL_BEFORE_KEYWORD = '&query='
13     TARGET_URL_REST = '&check_news=1&more=1&sorting=3&search_date=1&v1=&v2=
14     =&range=3'
15
16
17     # 기사 검색 페이지에서 기사 제목에 링크된 기사 본문 주소 받아오기
18     def get_link_from_news_title(page_num, URL, output_file):
19         for i in range(page_num):
20             current_page_num = 1 + i*15
21             position = URL.index('=')
22             URL_with_page_num = URL[: position+1] + str(current_page_num) ₩
23                 + URL[position+1 :]
24             source_code_from_URL = urllib.request.urlopen(URL_with_page_num)
25             soup = BeautifulSoup(source_code_from_URL, 'xml',
26                 from_encoding='utf-8')
27             for title in soup.find_all('p', 'tit'):
28                 title_link = title.select('a')
29                 article_URL = title_link[0]['href']
30                 get_text(article_URL, output_file)
31
32
33     # 기사 본문 내용 긁어오기 (위 함수 내부에서 기사 본문 주소 받아 사용되는 함수)
34     def get_text(URL, output_file):
35         source_code_from_url = urllib.request.urlopen(URL)
```

```

36     soup = BeautifulSoup(source_code_from_url, 'lxml', from_encoding='utf-8')
37     content_of_article = soup.select('div.article_txt')
38     for item in content_of_article:
39         string_item = str(item.find_all(text=True))
40         output_file.write(string_item)
41
42
43 # 메인함수
44 def main(argv):
45     if len(argv) != 4:
46         print("python [모듈이름] [키워드] [가져올 페이지 숫자] [결과 파일명]")
47         return
48     keyword = argv[1]
49     page_num = int(argv[2])
50     output_file_name = argv[3]
51     target_URL = TARGET_URL_BEFORE_PAGE_NUM + TARGET_URL_BEFORE_KEYWORD ₩
52                 + quote(keyword) + TARGET_URL_REST
53     output_file = open(output_file_name, 'w')
54     get_link_from_news_title(page_num, target_URL, output_file)
55     output_file.close()
56
57
58 if __name__ == '__main__':
59     main(sys.argv)
60

```

위 코드는 동아일보에서 특정 키워드를 가지는 신문기사를 대량으로 크롤링하기 위한 크롤러 모듈입니다. 전체적인 모듈의 동작 과정은 다음과 같습니다.

첫째, 특정 키워드를 통해 신문 기사를 검색해 관련된 기사 목록을 얻어 옴
둘째, 신문사의 기사 목록 페이지의 URL주소 패턴을 분석해, 반복문으로 여러 목록 페이지를 돌며 올라와 있는 기사 제목에 연결된 모든 링크 주소(기사 본문 URL 주소)를 얻어 옴
셋째, 얻어낸 기사의 링크 주소 하나하나에 접근해, 기사 본문 내용이 담긴 HTML 요소를 찾아 해당 요소만 긁어와 본문 내용을 하나의 파일에 저장

그럼 아래에서 한줄한줄 코드를 살펴보며 자세히 설명하도록 하겠습니다.

```

1     import sys
2     from bs4 import BeautifulSoup
3     import urllib.request
4     from urllib.parse import quote

```

해당 모듈에서 크롤링을 위해 사용할 라이브러리를 임포트한 부분입니다.

해당 모듈을 실행할 때, 터미널에서 사용자로부터 인자를 받기 위해 'sys.argv'를 사용하려고 'sys'를 임포트했고, 타겟 URL에 요청을 보내고 응답을 받기 위해 'urllib'을, 받은 응답(HTML 코드)를 파싱하기 위해

'BeautifulSoup'을 임포트했습니다.

이전 예제와는 다르게 'urllib.parse'에서 'quote'함수를 임포트했는데, 'quote'는 'urlopen'에서 인자로 사용되는 URL주소(이하 타겟 주소)에 한글(UTF_8)이 포함되었을 때, 이를 아스키(ASCII)형식으로 바꿔주기 위한 함수입니다.

이 내용은 해당 함수를 사용할 때, 다시 한번 자세히 설명하겠습니다.

1	TARGET_URL_BEFORE_PAGE_NUM = " http://news.donga.com/search?p= "
2	TARGET_URL_BEFORE_KEYWORD = '&query='
3	TARGET_URL_REST = '&check_news=1&more=1&sorting=3&search_date=1&v1=&v2=&range=3'

이 부분은 'urlopen'으로 요청을 보낼 타겟 주소를 세 부분으로 나누어 상수에 할당한 부분입니다.

타겟 주소를 왜 세 부분으로 나누어놓았는지 알기 위해선 'urlopen'으로 처음 접근할 동아일보의 기사 검색 페이지(이하 타겟 페이지)를 찾아 해당 페이지의 URL패턴을 찾아내야 합니다.



위는 동아일보 메인 페이지입니다.

URL 패턴을 찾기 위해 검색 키워드로 '사드'를 사용해 우측 상단 검색 공간에서 검색해보겠습니다.

통합검색 뉴스 스포츠동아 매거진 포토 상세검색

정렬 > 정확도순 최신순 오래된순 범위 > 전체 제목 동아일보 기자명 기간 > 전체 1주 1개월 1년

파워링크 NO

클릭초이스 등록 >

[사드레이싱 린스물](#) www.rinseall.com

레이싱 유전자를 이어받은 고성능 프리미엄오일, 엔진수명 및 연비를 극한까지

이슈연재 (총 1 건 검색)

☒ 최신순 ☐ 오래된순 ☐ 정확도순[사드 체계 한반도 배치](#) [이슈] 2015-03-17 00:00

연동아일보 (총 2234 건 검색)

☒ 최신순 ☐ 오래된순 ☐ 정확도순[한미군 "北 SLBM 실전배치될 경우 우리 대응능력 뚝 떨어진다"](#) 2016-08-29 14:01

...도한팩트리프트와 중거리 지대공유도무기(M-SAM)의 성능개량, 장거리 지대공유도무기(L-SAM)의 연구개발배치 가속화, 주한미군 사드(고고도미사일방어체계-THADD) 배치를 신속하게 추진해 요격 능력을 강화한다. 아울러 자항기력과 장

[뉴스1 > 사회](#)[사드 배치 최종호보지, '성산포대 뒤편'에 대결 유력](#) 2016-08-29 11:57

...특히 사드를 운용할 주한미군이 골프장을 이용할 수 있다는 점에서 긍정적인 평가를 받고 있는 것으로 알려졌다. 국방부는 이미 실무선에서 제3후보지로 거론되는 성주군 초전면 롯데 성주골프장 인근 지역을 세 차례나 시사했다. 다만 성주골프

[뉴스1 > 사회](#)[北 "핵경수환 배치" 핵무장론 재검토...이번엔 탄력받나](#) 2016-08-29 11:40

...이에 더해 사드(고고도미사일방어체계-THADD) 한성 달론을 채택하는 등 안보에 대한 집권여당의 안보 강경기조가 심화하는 양상이다. '핵무장론'에는 한작전직 원내대표가 팔을 걷어붙였다. 정찬석 원내대표는 이날 국회에서 열린 최고위원회

[뉴스1 > 정치](#)

많이 본 뉴스 | 댓글이 핫한 뉴스

종합 | 정치 | 국제 | 사회 | 스포츠 | 연예

- 1 [속보] 대우조선 호화 접대 의혹 조선일보...
- 2 부산 30대 부부 실종 미스터리
- 3 성남 판교-여주, 2시간-48분으로...경강...
- 4 진주 건물 자붕 붕괴 사고 생존자 '아프자'
- 5 탤런트 김동현, 사기 혐의로 또 피소

- 6 "30대 여강사, 13세 제자와 성관계 합의했...
- 7 '최성국 승부조작' 파문때 중자본 끌어들인...
- 8 "그때 보관에 갔어야 했다"...최경희 이대...

사드로 검색한 결과 페이지입니다. 우리는 정확히 사드와 관련된 신문 기사만 필요하기 때문에 본 화면에서 주황색으로 칠해진 카테고리 탭을 뉴스로 한정 짓고, 카테고리 탭 바로 밑의 검색필터에서 정렬은 정확도순으로 범위는 동아일보로 기간은 전체로 다시 한번 검색을 설정하도록 하겠습니다.

위 화면은 더보기 버튼을 누른 화면입니다.

이전과는 다르게 많은 기사들이 한 페이지당 15개씩 게시되어 나타나는 것을 알 수 있습니다.

(한 사진에 해당 웹페이지 화면은 다 담을 수 없어 하단 부분만 사진으로 담았습니다.)

우리는 이 화면 뿐 아니라 원하는 갯수 만큼의 기사 목록 페이지(2페이지, 3페이지 ...)에 있는 각각의 기사에 접근해 본문 내용을 수집해야 합니다.

기사 내용을 수집하기 위해선 실제 기사 페이지로 접근해야 하기 때문에 파란색으로 링크처리가 된 기사의 제목에서 연결된 기사 본문 URL주소를 뽑아와야 합니다.

그럼 먼저 이 페이지의 URL주소를 한번 살펴보겠습니다.

http://news.donga.com/search?check_news=1&more=1&sorting=3&range=3&query=사드

사진 속 URL주소창을 보게 되면 위 화면의 URL주소는 위와 같습니다.

'check_news=1'은 뉴스로 한정짓는 것을, 'more=1'은 더보기 버튼을 누른 상태임을, 'sorting=3'과 'range=3'은 각각 정확도순 정렬이며, 전체기간검색임을, 마지막으로 'query=사드'는 우리가 검색한 키워드임을 지레 짐작 할 수 있습니다.

하지만 아직도 이 페이지가 우리가 크롤링할 타겟 페이지가 되기엔 부족합니다.

우리는 1번 페이지의 15개의 기사만 가져올 것이 아니라 2번 페이지, 3번 페이지 등등 원하는 페이지 수 만큼 기사를 크롤링해야 하기 때문입니다.

그러나 이상하게도 위 URL주소에는 페이지에 해당 하는 내용이 있어 보이지 않습니다.

사실 페이지에 관한 내용은 숨겨져 있습니다.

하단의 페이지 숫자를 눌러 2번째 페이지로 들어가 보겠습니다.

news.donga.com/search?p=16&query=사드&check_news=1&more=1&sorting=3&search_date=1&v1=&v2=&range=3

...미국 정부는 최근 북한의 핵실험과 미사일 발사 등 위협에 대응하기 위해 중대사(中大史)를 제정하고, 이를 통해 사드(THAAD) 배치에 대한 논의를 진행하고 있다. 특히 중대사(中大史)는 미국 중대사(中大史)인민해방군 기관지(解放)에 보도가 한 번도 배치된 사드(THAAD) 배치에 대해 1시간 이내에 성명할 수 있다고 주장했다고 보도했다...

동아일보 > 정치 | 윤상호 기자, 이유진 기자

한대표까지 찾아가... 중대사의 '사드' 반대 2016-02-24 03:00

...추귀(追歸) 주한 중국대사는 23일 국회로 더불어민주당 김중권 비상대책위원회 대표를 예방한 자리에서 '사드' 배치는 중국 안보에 큰 영향을 미친다. 이런 (사드) 문제가 중국의 안보 이익을 훼손한다면 양국 관계를 발전시키는 노력은 순식...

동아일보 > 정치 | 윤상호기자전문 기자, 조승호 기자

한 "사드는 자국과 관련"... 외교부, 중대사 초지 2016-02-25 03:00

...김중권 외교부 차관보는 더불어민주당을 예방한 자리에서 '사드' 반대 '한중 관계 파탄 가능성' 등의 언급으로 외교적 결례를 범한 추귀(追歸) 주한 중국대사를 24일 외교부로 초지(초지)했다. 주 대사는 이 자리에서 더민주당 방문 경위와 실제 언급 내용 등을 ...

동아일보 > 정치 | 손호주 기자, 정택동 기자, 조승호 기자

美, 사드 배치 속도조절... 안보리 대목제재 2016-02-26 03:00

...미국에 대통령까지 나서 '국익에 따라 배치를 결정하겠다'며 필요성을 강조한 사드가 미국 사이의 협상카드로 활용되고 있는 것 아니냐는 지적이 나온다. 미국 고위 당국자가 잇달아 한국을 방문하는 것도 공교롭다. 외교부는 25일 '미사일 리콜 및 미국무 동여'에 차관보가...

동아일보 > 정치 | 손호주 기자, 이승현 특파원, 조승호 기자

1 2 3 4 5 6 7 8 9 10 다음 >

사드

위 화면은 두번째 페이지로 넘어 온 화면입니다.

첫번째 화면의 URL 주소와는 달리 두번째 페이지의 URL주소가 달라진 것을 확인할 수 있습니다.

http://news.donga.com/search?p=16&query=사드&check_news=1&more=1&sorting=3&search_date=1&v1=&v2=&range=3

기존의 'query=사드' 위치가 앞쪽으로 이동해 왔으며 페이지숫자로 추정되는 'p=16'이 나타났습니다. 16의 숫자가 페이지를 어떤 식으로 정의하는지 알아보기 위해 다시 여러 페이지를 눌러 URL 주소를 찾아 본 결과 아래와 같이 변했습니다.

1번 페이지: http://news.donga.com/search?p=1&query=사드&check_news=1&more=1&sorting=3&search_date=1&v1=&v2=&range=3
 2번 페이지: http://news.donga.com/search?p=16&query=사드&check_news=1&more=1&sorting=3&search_date=1&v1=&v2=&range=3
 3번 페이지: http://news.donga.com/search?p=31&query=사드&check_news=1&more=1&sorting=3&search_date=1&v1=&v2=&range=3
 4번 페이지: http://news.donga.com/search?p=46&query=사드&check_news=1&more=1&sorting=3&search_date=1&v1=&v2=&range=3

URL 주소가 변화하는 것을 살펴보면 페이지가 증가할 때마다 'p'의 값이 15씩 더해지는 것을 알 수 있습니다.

한 페이지에 나타나는 기사 수 15개인 것으로 보아 한 페이지가 증가할 때마다 p의 값을 15씩 늘려 페이지를 구분한 것을 알아낼 수 있습니다.

그럼 다시 한번 타겟 주소를 상수화한 부분을 살펴보겠습니다.

```
11 TARGET_URL_BEFORE_PAGE_NUM = "http://news.donga.com/search?p="
12 TARGET_URL_BEFORE_KEYWORD = '&query='
13 TARGET_URL_REST = '&check_news=1&more=1&sorting=3&search_date=' ₩
14 '1&v1=&v2=&range=1'
```

'사드'를 통해 검색된 기사 목록 전체를 둘러보기 위해선 'query=사드'로 설정되어야 하며 페이지 번호를 'p=1', 'p=16', 'p=31'... 처럼 원하는 페이지를 나타내는 수 만큼 변경시켜가며 전부 둘러봐야 하는 것을 알 수 있습니다.

따라서 위 처럼 페이지수를 나타내는 'p='부분과 검색 키워드를 나타내는 '&query='부분의 정보를 사용자로부터 입력받아 하나의 타겟 주소로 결합하기 위해서 타겟 주소를 세 부분으로 나눈 것입니다.

그럼 타겟 주소가 어떻게 결합되어 사용되는지 알아보기 위해 크롤러 모듈에서 메인함수를 먼저 살펴보겠습니다.

```
42 # 메인함수
43 def main(argv):
44     if len(argv) != 4:
45         print("python [모듈이름] [키워드] [가져올 페이지 숫자] [결과 파일명]")
46         return
47     keyword = argv[1]
48     page_num = int(argv[2])
49     output_file_name = argv[3]
50     target_URL = TARGET_URL_BEFORE_PAGE_NUM + TARGET_URL_BEFORE_KEYWORD ₩
51         + quote(keyword) + TARGET_URL_REST
52     output_file = open(output_file_name, 'w')
53     get_link_from_news_title(page_num, target_URL, output_file)
54     output_file.close()
55
56
```

```

57     if __name__ == '__main__':
58         main(sys.argv)

```

위 코드는 메인함수 부분입니다.

'sys.argv'를 통해 사용자로 부터 검색 키워드, 페이지 숫자, 결과 파일명을 각각 'keyword', 'page_num', 'output_file_name'에 입력받습니다.

'page_num'은 정수형으로 쓰여야 하므로 int형으로 변환해서 사용합니다.

이후, target_URL 변수에 모듈 위에서 정의해놓은 URL상수들을 사용자로 부터 입력받은 정보와 결합시킵니다.

이때, 'TARGET_URL_BEFORE_PAGE_NUM'뒤에 사용자로 부터 입력받은 페이지 숫자를 결합하지는 않고 검색 키워드인 'keyword'만 'TARGET_URL_BEFORE_KEYWORD'뒤에 'quote'(urllib.parse의 메소드)메소드만 사용해 결합합니다.

(페이지 숫자를 결합하지 않는 이유는 아래에서 설명하겠습니다.)

'quote' 메소드를 'keyword'에 사용하는 이유는 우리가 검색할 때 사용하는 언어는 한글('UTF-8')이기 때문입니다.

URL주소에는 'ASCII' 표현 방식 이외의 문자표기법은 사용될 수 없기 때문에 '사드'라는 'UTF-8' 방식의 문자를 'ASCII' 방식으로 변환해야하기 위해 'quote'메소드를 'keyword'에 사용했습니다.

그렇게 완성된 타겟 주소(사실 페이지 숫자가 아직 포함되지 않았으므로 완전한 형태의 타겟 주소는 아닙니다.)를 'get_link_from_news_title'함수를 이용해 기사를 크롤링해와 파일로 저장합니다.

```

16     # 기사 검색 페이지에서 기사 제목에 링크된 기사 본문 주소 받아오기
17     def get_link_from_news_title(page_num, URL, output_file):
18         for i in range(page_num):
19             current_page_num = 1 + i*15
20             position = URL.index('=')
21             URL_with_page_num = URL[: position+1] + str(current_page_num) + \
22                 + URL[position+1 :]
23             source_code_from_URL = urllib.request.urlopen(URL_with_page_num)
24             soup = BeautifulSoup(source_code_from_URL, 'lxml',
25                                 from_encoding='utf-8')
26             for title in soup.find_all('p', 'tit'):
27                 title_link = title.select('a')
28                 article_URL = title_link[0]['href']
29                 get_text(article_URL, output_file)

```

위 함수는 메인함수에서 크롤링하기 위한 함수인 'get_link_from_news_title'함수 입니다.

인자로 는 크롤링해 올 기사 목록 수인 page_num과, 타겟 주소인 URL, 그리고 본문 내용을 저장할 결과파일명인 'output_file'을 인자로 받습니다.

함수 안 첫번째 반복문은 여러 기사 목록을 돌며 기사 제목에 링크된 기사 본문 주소(URL주소)를 얻어 오는 부분입니다.

이 반복문에서는 기사를 굼어올 페이지 수만큼 코드를 반복합니다.

'current_page_num'에 'i'를 이용해서 굼어올 페이지 수만큼의 현재 페이지를 만듭니다.

(i=0 일때 current_page_num=1, i=1 일때 current_page_num=16, i=2 일때 current_page_num=31 ...)

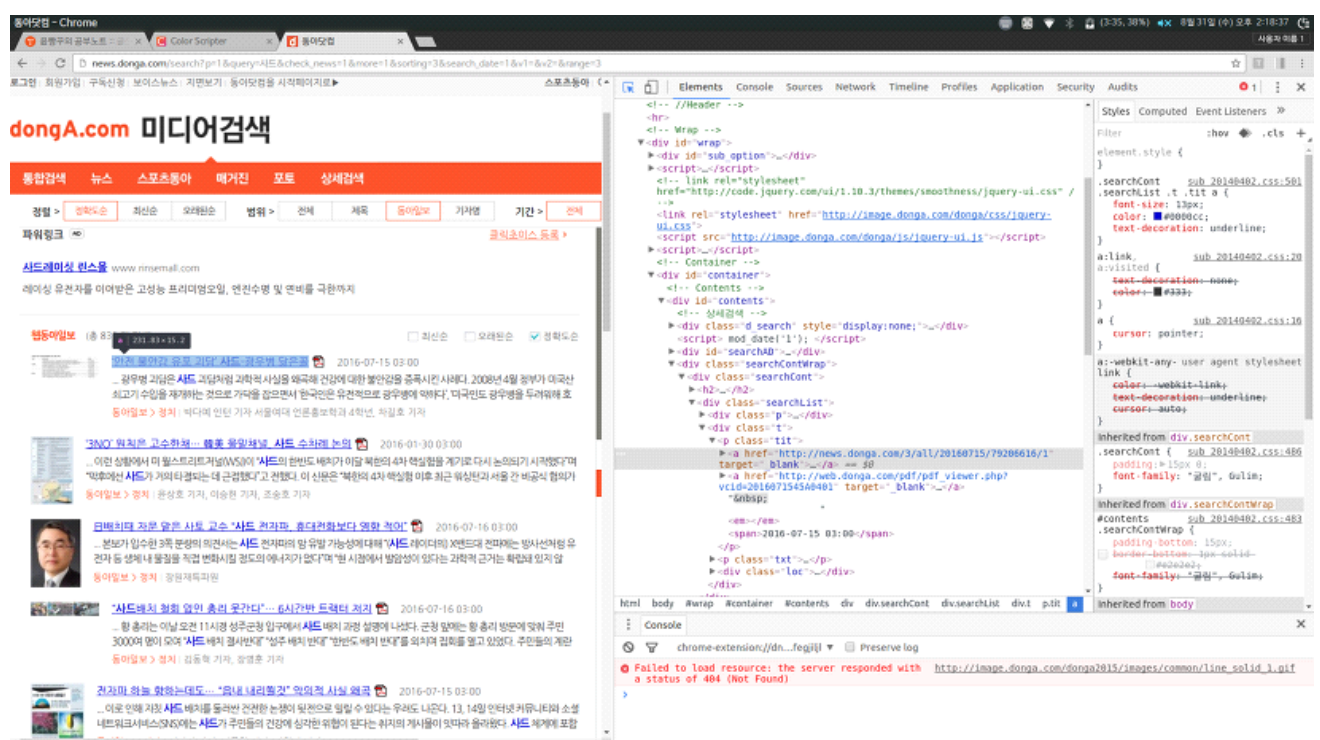
그 후, 인자로 받은 URL에서 첫번째로 '='문자가 나오는 위치를 'position'에 할당한 후, 해당 'position'뒤에 'current_page_num'을 스트링으로 변환한 후 삽입합니다.

(21~22번째 줄 코드가 이해가 가지 않는다면 문자열 슬라이스를 검색해 살펴보시길 바랍니다.)

URL에서 처음으로 '='문자가 나오는 위치는 아까 위에서 URL 주소에 마저 결합하지 않았던 페이지 숫자를 나타내는 부분이기 때문에 위와 같은 방법으로 완전한 타겟 주소를 만들수 있으며 반복문을 통해 1번 페이지부터, 원하는 페이지(예를 들어 원하는 페이지 수가 10이면 i의 최대값은 9가 되고, 그때의 'current_page_num'은 136이 됩니다.)까지의 모든 URL주소를 만들어 낼 수 있습니다.

이렇게 만들어진 'URL_with_page_num'을 이용해 'urlopen'으로 요청을 하고 응답을 받아 'BeautifulSoup'객체인 'soup'을 만듭니다.

이제부터는 모든 기사 목록에 올라온 기사 제목에서 기사 본문 주소(URL주소)를 추출해올 수가 있습니다. 하단의 26번째 줄 부터 시작되는 반복문은 기사 제목에 링크된 기사 본문 주소(URL주소)를 얻어오기 위한 부분입니다. 이를 설명하기 앞서 다시 한번 동아일보 기사 검색 페이지 화면을 보도록 하겠습니다.



'사드'로 검색한 기사 목록 첫번째 페이지입니다. 크롬 개발자 도구를 통해 첫번째 기사의 제목(링크)부분이 어딘지 찾아봤습니다.

```
<p class="tit">
  <a href="http://news.donga.com/3/all/20160715/79206616/1"
  target="_blank">...</a> == $0
  <a href="http://web.donga.com/pdf/pdf_viewer.php?
  vcid=2016071545A0401" target="_blank">...</a>
  "&nbsp;";
</p>
```

class가 'tit'인 p태그 안의 첫번째 a태그에 연결된 URL주소가 해당 기사 본문 URL이 포함된 것을 알 수 있습니다.

직접 더 살펴본 결과, 2번째 기사, 3번째 기사 15번째 기사 제목 모두 연결된 기사 본문 URL주소가 위와 같은 형식이었습니다.

따라서 우리는 위에서 만든 'soup'객체에서 class='tit'인 p태그를 모두 뽑아와 그 안에 있는 첫번째 a태그의

'href'의 내용을 가져온다면 모든 기사의 내용을 크롤링할 수 있습니다.
그럼 'get_link_from_news_title'함수의 하단 반복문을 다시 보도록 하겠습니다.

```
26         for title in soup.find_all('p', 'tit'):  
27             title_link = title.select('a')  
28             article_URL = title_link[0]['href']  
29             get_text(article_URL, output_file)
```

위에서 생성한 'soup'객체에서 'find_all' 메소드를 통해 class가 'tit'인 p태그를 모두 가져와 반복문을 통해 'title'에 하나씩 할당했으며 'select'메소드를 통해 모든 a태그를 'title_link'에 저장했습니다.
('select' 메소드는 인자로 주어지는 태그를 모두 가지고 오며, 해당 객체는 인덱스로 태그 하나하나에 접근할 수 있습니다. 자세한 사항은 [BeautifulSoup 문서](#)를 참조하시기 바랍니다.)

기사 본문 URL 주소는 p태그 안 첫번째 a태그에 저장되어 있으므로 'title_link'의 0번 인덱스로 첫번째 a태그에 접근했으며 키 값으로 'href'를 이용해 'href'로 연결된 URL주소를 'article_URL'에 저장했습니다.
(역시나 BeautifulSoup 객체의 자세한 사용법은 [BeautifulSoup 문서](#)를 참조하시기 바랍니다.)

2개의 반복문을 통해 최종적으로 얻어진 기사 본문 URL주소를 'get_text'함수에 결과 파일명과 함께 전달합니다.

'get_text'메소드는 실제로 각 기사의 본문 내용을 크롤링해오는 함수입니다.

```
32     # 기사 본문 내용 긁어오기 (위 함수 내부에서 기사 본문 주소 받아 사용되는 함수)  
33     def get_text(URL, output_file):  
34         source_code_from_url = urllib.request.urlopen(URL)  
35         soup = BeautifulSoup(source_code_from_url, 'xml', from_encoding='utf-8')  
36         content_of_article = soup.select('div.article_txt')  
37         for item in content_of_article:  
38             string_item = str(item.find_all(text=True))  
39             output_file.write(string_item)  
40
```

위는 기사 본문 주소를 받아 본문 내용을 긁어오는 'get_text'함수 입니다.

[이전 글](#)에서 네이버뉴스를 긁어오기 위해 사용했던 함수와 큰 차이는 없습니다.

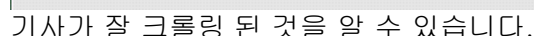
이 함수는 기사 URL 주소를 통해 응답을 받아 'BeautifulSoup'객체를 만들고, 본문을 추출해 결과 파일에 쓰는 동작을 합니다.

다만 이 함수에서 다른 점이 있다면, 동아일보에서는 기사의 본문 내용이 class가 'article_txt'인 div태그안에 포함되어 있었기 때문에 'select'메소드를 통해 class가 'article_txt'인 div태그안 텍스트 요소만 뽑아온 점입니다.

다시 한번 두 함수를 정리해보면 다음과 같습니다.

'get_link_from_title'함수를 통해 특정 키워드로 검색한 기사 목록의 모든 기사 주소들을 추출해오며 해당 함수 내부에서 추출한 기사 주소들을 이용해 'get_text'함수를 호출해 결과 파일에 기사 본문 내용을 누적해 저장하는 방식으로 두 함수는 동작하게 됩니다.

터미널을 통해 크롤러 모듈을 실행시켜 사드 관련 기사를 10페이지(150개)를 크롤링해 결과 파일을 'result_articles.txt'에 저장했습니다.
그 저장 결과는 다음과 같습니다.



해당 코드 뿐 아니라 불필요한 문자는 이전글에서 설명한 텍스트 정제 모듈로 제거해서 분석에 사용하시면 됩니다.

다만 한겨레신문에서 사용하는 별도의 URL패턴이 있을 것이며 기사 본문 내용을 구성하는 HTML코드 또한 동아일보와 다를 것입니다.

한겨레신문을 위한 크롤러 모듈은 아래의 코드를 참조하시기 바랍니다.


```

1  """ 한겨레 신문 특정 키워드를 포함하는, 특정 날짜 이전 기사 내용 크롤러(정확도순 검색)
2      python [모듈이름] [키워드] [가져올 페이지 숫자] [가져올 기사의 최근 날짜]
3      [결과 파일명.txt]
4      한페이지에 10개
5      """
6
7
8  import sys
9  from bs4 import BeautifulSoup
10 import urllib.request
11 from urllib.parse import quote
12
13 TARGET_URL_BEFORE_KEYWORD = 'http://search.hani.co.kr/Search?command=query&' W
14     'keyword='
15 TARGET_URL_BEFORE_UNTIL_DATE = '&media=news&sort=s&period=all&datefrom=' W
16     '2000.01.01&dateto='
17 TARGET_URL_REST = '&pageseq='
18
19
20 def get_link_from_news_title(page_num, URL, output_file):
21     for i in range(page_num):
22         URL_with_page_num = URL + str(i)
23         source_code_from_URL = urllib.request.urlopen(URL_with_page_num)
24         soup = BeautifulSoup(source_code_from_URL, 'xml',
25                             from_encoding='utf-8')
26         for item in soup.select('dt > a'):
27             article_URL = item['href']
28             get_text(article_URL, output_file)
29
30
31 def get_text(URL, output_file):
32     source_code_from_url = urllib.request.urlopen(URL)
33     soup = BeautifulSoup(source_code_from_url, 'xml', from_encoding='utf-8')
34     content_of_article = soup.select('div.text')
35     for item in content_of_article:
36         string_item = str(item.find_all(text=True))
37         output_file.write(string_item)
38
39
40 def main(argv):
41     if len(sys.argv) != 5:
42         print("python [모듈이름] [키워드] [가져올 페이지 숫자] "
43             "[가져올 기사의 최근 날짜] [결과 파일명.txt]")
44         return
45     keyword = argv[1]
46     page_num = int(argv[2])
47     until_date = argv[3]
48     output_file_name = argv[4]
49     target_URL = TARGET_URL_BEFORE_KEYWORD + quote(keyword) W
50         + TARGET_URL_BEFORE_UNTIL_DATE + until_date + TARGET_URL_REST
51     output_file = open(output_file_name, 'w')
52     get_link_from_news_title(page_num, target_URL, output_file)
53

```

```
54     output_file.close()
55
56     if __name__ == '__main__':
57         main(sys.argv)
58
```